Lab 2: Phase Plane Analysis of Biochemical Systems

Stream 1: Cell Signalling

August 14, 2026

1 Goal

To model biochemical reaction systems using ordinary differential equations (ODEs), solve these systems numerically, and build and interpret phase portraits step-by-step to understand complex dynamic behaviors.

2 Reading Material

Keener, J., & Sneyd, J. (2025). *Mathematical Physiology* (3rd ed.). Springer.

- **Chapter 1: Biochemical Reactions**, particularly Section 1.4 on Chemical Kinetics and Section 1.5 on Glycolysis and Glycolytic Oscillations.

3 Guided Example: Building a Phase Portrait for Michaelis-Menten Kinetics

You will build a complete phase portrait for the dimensionless Michaelis-Menten system. The script `lab2_mm_guided.py` is structured to match the steps below.

The system equations are:

$$\begin{aligned} \frac{ds}{dt} &= \alpha c - s(1-c), \\ \frac{dc}{dt} &= -\beta(\alpha c - s(1-c)) - \varepsilon c, \end{aligned} \tag{1}$$

with parameters $\alpha = 0.6$, $\beta = 1.25$, and $\varepsilon = 0.4$.

Part A: Solving the ODEs and Plotting a Trajectory

First, solve the system numerically and plot the solution.

i. **Define the System in Python:** The script defines a function for the ODEs that takes state variables, parameters and initial conditions.

ii. **Solve Numerically:** Use the `compute` function to integrate the system from an initial condition, for example, $(s_0, c_0) = (1, 0)$.

iii. **Plot the Results:** The script generates two plots using Matplotlib's `ax.plot()` function: a time series showing how $s(t)$ and $c(t)$ evolve, and a phase plane plot showing the trajectory of $(s(t), c(t))$.

Part B: Adding the Vector Field

Next, visualize the flow of the system at all points in the phase plane.

i. **Plot the Field:** The script uses the `plot_PP_vf` function from the phaseplane toolbox of PyDSTool to plot the vector fields.

Note: The function has no option to normalise. However, it has a `scale_exp` variable which rescales the size of arrows in case of disparate scales in the vector field. Larger values of scale magnify the arrow sizes.

Part C: Adding Nullclines and the Steady State

Finally, add the nullclines to complete the portrait.

i. **Define Nullclines:** The nullclines are curves where one derivative is zero. Solve the equations $\frac{ds}{dt} = 0$ and $\frac{dc}{dt} = 0$ for one variable in terms of the other. The `find_nullclines` function from the phaseplane toolbox does this for you.

ii. **Plot the Nullclines:** The script plots these curves using `ax.plot()`, this time with a dashed line style (e.g., `'r--'`). Their intersection marks the system's steady state(s).

iii. **Mark the Steady State:** The script explicitly marks the steady state at $(0, 0)$ using `ax.plot()` with a star marker (`'m*'`).

Part D: Report and Discussion

(a) **Create a Report:** Write a brief LaTeX report of this guided example. Replicate the introduction to this problem and then include the plots you generated in Parts A, B, and C. Ensure that each plot has a descriptive caption.

(b) **Discussion:** Discuss whether there exists a further *one*-dimensional approximation to the dynamics of the substrate concentration $s(t)$ when ε becomes sufficiently small, and argue where such an approximation would be valid in phase space.

Hint: Modify the script to plot the nullclines for a sufficiently small ε value (e.g., $\varepsilon = 0.01$) and compare them. Include this new plot in your report to support your argument.

4 Exercise: Modeling the 'Selkov' Glycolytic Oscillator

Now, apply what you've learned. This version of the Selkov model is a simplified representation of glycolytic oscillations:

$$\frac{dx}{dt} = -x + ay + x^2 y$$
$$\frac{dy}{dt} = b - ay - x^2 y\,.$$

It is a dimensionless model where x denotes the dimensionless substrate concentration and y denotes the dimensionless product concentration, and the dimensionless parameters a and b are non-negative. For a biochemically more realistic glycolytic oscillator model, please look at Section 1.5 in the textbook.

Task: Create time traces and the phase portrait for the Selkov model

Your task is to adapt the script `lab2_mm_guided.py` to create a new Python script that analyses the Selkov model.

i. **Modify the Code:**

- Change the ODE function to implement the Selkov equations.
- Update the parameters to $a = 0.1$ and $b = 0.6$.
- Use the initial condition $x(0) = 0.75, y(0) = 1$ and simulate for a long enough time (e.g., $t = 100$) to see the long-term behavior.
- Use a second initial condition $x(0) = 1.5, y(0) = 1.5$ and simulate for a long enough time (e.g., $t = 100$) to see the long-term behavior.
- Update the code for plotting the nullclines and the steady state according to the Selkov model's equations.

ii. **Generate Plots:** Your final Python script should produce:

- A time series plot for $x(t)$ and $y(t)$.
- A complete phase portrait showing the trajectory, vector field, nullclines, and the (unstable) steady state.

- Make sure that the plotting range for each plot is adjusted accordingly.

iii. **Analyse:** Based on your plots, describe the long-term behavior of the system. What feature of the phase portrait corresponds to sustained oscillations? Discuss with your peers!

Advanced Task[1]: Continuation analysis

This will show how the system's behavior changes as the parameter b (the input flux) is varied. Here, we set $b = 0$.

i. **Find the Steady State:** Before running the continuation, you must find a steady state to start from. This was found previously using the `find_fixedpoints` function.

ii. **Run Continuation:** Create a PyCont instance of the ode system we defined using `ContClass` function with curve type 'EP-C' (equilibrium curve).

   – Set `freepars` to `['b']`.
   – Set `MaxNumPoints` to 200
   – Set `MaxStepSize` to 0.001
   – Set `initpoint` to the steady state
   – Set `LocBifPoints` to 'all' or 'H'

iii. **Analyse the bifurcation diagram:** The resulting plot shows the steady-state value of x as a function of b.

   - A black line indicates a stable steady state.
   - A dotted-black line indicates an unstable steady state.
   - PyDSTool will automatically detect and label bifurcation points. You should see two points labeled **H** for a **Hopf bifurcation** (if verbosity is set to 2). This is the point where the steady state loses stability and a limit cycle (oscillation) is born.
     – Create a new PyCont instance of curve type 'LP-C' (limit point curve) with same configurations as the previous curve.
     – Set `MaxNumPoints` to 50
     – Set `initpoint` to 'EQ1:H1'. This means from the "EQ1" curve pick the point "H1" as the starting point. The curves name is not always "EQ1", it is what ever you give it.
     – The two branches of purple curve appear represent the minimum and maximum values of the stable limit cycle.

iv. **Discussion:** Compare with the description in the textbook section 1.5! What are the main differences between the bifurcation diagrams of the simplified version here and the one shown in the textbook?

[1] We will discuss bifurcation theory at a later stage in this project unit. For the moment, it is all about getting used to the software package.